

ENGENHARIA DE COMPUTAÇÃO

Confiabilidade, Segurança de Sistemas e Ergonomia

Rotinas Matemáticas de Proteção e Ocultação de Dados

Hashing com Salt · Criptografia Simétrica AES · Esteganografia LSB

Integrantes do grupo

Lucas Gabriel Bento Correa — RA: 220020281

Leonardo Fernandes Fanhani — RA: 220020517

Kauã Enrico Pasti Altran — RA: 220019627

Henrique Pignato — RA: 220020592

Luke Gabriel dos Santos Belatine — RA: 220020097

Linguagem utilizada: Python 3.13

Setembro de 2026

1. Objetivo

Este relatório descreve a implementação de três rotinas matemáticas de proteção e ocultação de dados, desenvolvidas em Python: (1) um módulo de hashing para simular o cadastro seguro de usuários, (2) um módulo de criptografia simétrica para cifrar dados financeiros hipotéticos e (3) um módulo de esteganografia para ocultar uma frase confidencial dentro de uma imagem. O objetivo é demonstrar, na prática, os conceitos de confidencialidade, integridade e ofuscação de informação estudados na disciplina.

O código completo está organizado em um repositório Git, com testes automatizados (pytest) que validam o funcionamento de cada módulo. A linguagem escolhida foi o Python 3.13, pela disponibilidade de bibliotecas robustas e amplamente auditadas para criptografia (cryptography) e manipulação de imagens (Pillow), além do módulo sqlite3 já incluso na biblioteca padrão.

Estrutura do projeto

```

.
├── demo.py                # executa a demonstração dos 3 módulos
├── requirements.txt        # dependências (cryptography, Pillow, pytest)
├── src/
│   ├── hashing/cadastro.py # Módulo 1 – hashing + salt + SQLite
│   ├── criptografia/cifra_aes.py # Módulo 2 – AES-256-GCM
│   └── esteganografia/lsb.py # Módulo 3 – esteganografia LSB
├── tests/                 # 15 testes automatizados (pytest)
├── assets/                # imagens geradas pela demonstração
└── data/                  # banco SQLite gerado em tempo de execução

```

2. Módulo de Hashing — Autenticação Segura

2.1 Fundamentação

O cadastro de usuários nunca grava a senha em texto puro. Para cada novo usuário é gerado um Salt criptográfico aleatório de 16 bytes, usando a função `secrets.token_bytes` (que utiliza uma fonte de aleatoriedade adequada para uso criptográfico, diferente do módulo `random`). Esse salt é injetado antes do processamento da senha pelo algoritmo PBKDF2-HMAC-SHA256, configurado com 200.000 iterações.

Optou-se por PBKDF2 (Password-Based Key Derivation Function 2) em vez de um único SHA-256 direto, porque o SHA-256 puro é uma função de hash muito rápida — projetada para verificar integridade de arquivos, não para proteger senhas — o que a torna vulnerável a ataques de força bruta e a tabelas pré-computadas (rainbow tables) quando aplicada isoladamente. O PBKDF2 aplica o HMAC-SHA256 de forma encadeada centenas de milhares de vezes, tornando cada tentativa de quebra de senha computacionalmente cara, mesmo para atacantes com hardware dedicado (GPU/ASIC).

O salt e o hash resultante (ambos em hexadecimal) são gravados em uma tabela `usuarios` de um banco SQLite local (`data/usuarios.db`), simulando a persistência de um sistema real. Na verificação de login, o salt armazenado é reutilizado para recalculer o hash da senha informada, e a comparação é feita com `secrets.compare_digest`, uma comparação em tempo constante que evita vazamento de informação por timing attacks.

2.2 Trecho de código relevante

```
def gerar_hash(senha: str, salt: bytes | None = None) -> tuple[bytes, bytes]:
    """Injeta um salt aleatorio (se nao for informado) e deriva o hash da senha."""
    if salt is None:
        salt = secrets.token_bytes(TAMANHO_SALT_BYTES) # 16 bytes aleatórios
    hash_senha = hashlib.pbkdf2_hmac(
        ALGORITMO_HASH, senha.encode('utf-8'), salt, ITERACOES # 200.000 iterações
    )
    return salt, hash_senha

def verificar_login(usuario: str, senha: str) -> bool:
    ...
    salt = bytes.fromhex(linha[0])
    hash_armazenado = linha[1]
    _, hash_calculado = gerar_hash(senha, salt)
    return secrets.compare_digest(hash_calculado.hex(), hash_armazenado)
```

2.3 Execução e evidências

A demonstração cadastra dois usuários (kaua e maria) com a mesma senha, tenta cadastrar um usuário duplicado (falha esperada) e realiza três tentativas de login: senha correta, senha incorreta e usuário inexistente.

```
=== Modulo de Hashing (Autenticacao Segura) ===
Usuario 'kaua' cadastrado com sucesso.
Usuario 'maria' cadastrado com sucesso.
Usuario 'kaua' ja existe.

Registros gravados no banco (nunca a senha em texto puro):
usuario='kaua' salt=1b6586941c6fe63e... hash=05a1c9d6fdd1655d9b997190a353a999...
usuario='maria' salt=27e0095a260e3be5... hash=04669fbbc993e0a2795c5fe382e790ed...

Prova de que a mesma senha gera hashes diferentes (salt aleatório):
hashes iguais? False

Tentativas de login:
kaua / senha correta -> True
kaua / senha errada -> False
usuario inexistente -> False
```

Observa-se que, mesmo com a senha idêntica ("SenhaForte#2024"), os hashes de kaua e maria são completamente diferentes — evidência direta de que o salt aleatório está sendo corretamente injetado antes do processamento. As tentativas de login confirmam que apenas a senha correta autentica o usuário.

3. Módulo de Criptografia — Cifra de Dados

3.1 Fundamentação

Este módulo cifra uma string contendo dados financeiros hipotéticos (número de cartão, validade, CVV, saldo em conta e chave PIX) usando criptografia simétrica AES-256, no modo de operação GCM (Galois/Counter Mode). Diferentemente de modos mais simples como o CBC, o GCM é um modo de cifragem autenticada (AEAD): além de garantir confidencialidade, ele produz uma tag de autenticação que garante integridade e autenticidade — qualquer alteração no texto cifrado, ou o uso de uma chave incorreta na decifragem, faz com que a operação falhe explicitamente (exceção InvalidTag), em vez de silenciosamente retornar dados corrompidos.

Por ser simétrica, a mesma chave de 256 bits é usada tanto para cifrar quanto para decifrar. A cada operação de cifragem é gerado um nonce (número usado uma única vez) aleatório de 96 bits, conforme recomendado pela especificação do AES-GCM, evitando o reuso de nonce — que comprometeria a segurança do esquema.

3.2 Trecho de código relevante

```
def gerar_chave() -> bytes:
    return AESGCM.generate_key(bit_length=256)

def cifrar(texto_claro: str, chave: bytes) -> DadosCifrados:
    aesgcm = AESGCM(chave)
    nonce = os.urandom(TAMANHO_NONCE_BYTES) # 96 bits, único a cada chamada
    texto_cifrado = aesgcm.encrypt(nonce, texto_claro.encode('utf-8'), None)
    return DadosCifrados(nonce=nonce, texto_cifrado=texto_cifrado)

def decifrar(dados: DadosCifrados, chave: bytes) -> str:
    aesgcm = AESGCM(chave)
    texto_claro = aesgcm.decrypt(dados.nonce, dados.texto_cifrado, None)
    return texto_claro.decode('utf-8')
```

3.3 Execução e evidências

```
=== Modulo de Criptografia (Cifra de Dados - AES-256-GCM) ===

Texto original:
  Cartao: 4111 1111 1111 1111 | Validade: 09/29 | CVV: 321 |
  Saldo em conta: R$ 15.320,50 | Chave PIX: kaua.financeiro@banco.com

Chave simetrica AES-256 (hex): cb3dac6332cbda8837c6dcf5a965bd8...

Texto cifrado (nonce + ciphertext, hex):
  96ec6f91005e3ed1221975d0de2b8c21b6c2b08170ee650110edf5ba0f3005f...

Texto decifrado com a chave correta:
  Cartao: 4111 1111 1111 1111 | Validade: 09/29 | CVV: 321 |
  Saldo em conta: R$ 15.320,50 | Chave PIX: kaua.financeiro@banco.com

Decifragem confere com o original? True

Tentando decifrar com uma chave incorreta (deve falhar)...
  Falhou como esperado (InvalidTag) -> integridade/autenticidade garantidas.
```

O texto decifrado com a chave correta é idêntico ao texto original, comprovando a corretude da rotina reversa. Ao tentar decifrar o mesmo texto cifrado com uma chave diferente, a biblioteca rejeita a operação com a exceção `InvalidTag`, demonstrando que o esquema também protege contra adulteração dos dados.

4. Módulo de Esteganografia — Ocultação LSB

4.1 Fundamentação

A esteganografia oculta a existência de uma mensagem, em vez de apenas cifrar seu conteúdo. A técnica utilizada foi a substituição do bit menos significativo (Least Significant Bit — LSB) dos canais de cor Vermelho, Verde e Azul (RGB) de cada pixel de uma imagem PNG.

A mensagem é primeiro convertida em bytes (UTF-8) e prefixada por um cabeçalho de 4 bytes contendo o seu tamanho — isso permite que a rotina de extração saiba exatamente quantos bytes ler, sem depender de um delimitador especial no meio do texto. Em seguida, cada bit do payload (cabeçalho + mensagem) substitui o bit menos significativo de um canal de cor, percorrendo os pixels da imagem da esquerda para a direita e de cima para baixo.

Como o bit menos significativo altera a intensidade de um canal de cor em, no máximo, uma unidade (por exemplo, de 200 para 201), a alteração é completamente imperceptível ao olho humano, apesar de ser trivialmente detectável e reversível por quem conhece o algoritmo — por isso a esteganografia é considerada uma técnica de ofuscação, e não de criptografia forte.

4.2 Trecho de código relevante

```
def esconder_mensagem(caminho_entrada, mensagem, caminho_saida):
    ...
    payload = _mensagem_para_payload(mensagem) # 4 bytes de tamanho + mensagem
    bits = _payload_para_bits(payload)
    ...
    for y in range(altura):
        for x in range(largura):
            r, g, b = pixels[x, y]
            canais = [r, g, b]
            for c in range(3):
                if indice < total_bits:
                    bit = int(bits[indice])
                    canais[c] = (canais[c] & ~1) | bit # substitui o LSB
                    indice += 1
            pixels[x, y] = tuple(canais)
```

4.3 Execução e evidências

```
=== Modulo de Esteganografia (Ocultacao LSB) ===

Capacidade maxima da imagem: 28796 bytes
Mensagem a esconder (50 bytes):
'Mensagem confidencial: reuniao as 15h no cofre 42.'

Tamanho do arquivo original: 3161 bytes
Tamanho do arquivo com mensagem: 3497 bytes
Maior diferenca de intensidade em qualquer canal: 1 (imperceptivel ao olho humano)

Mensagem extraida:
'Mensagem confidencial: reuniao as 15h no cofre 42.'
Extracao confere com o original? True
```

A extração recuperou exatamente a mesma mensagem que havia sido escondida, e a maior diferença de intensidade observada em qualquer canal de cor, entre a imagem original e a imagem modificada, foi de apenas 1 unidade — o que confirma que a imagem permanece visualmente idêntica à original, como mostram as duas imagens a seguir.

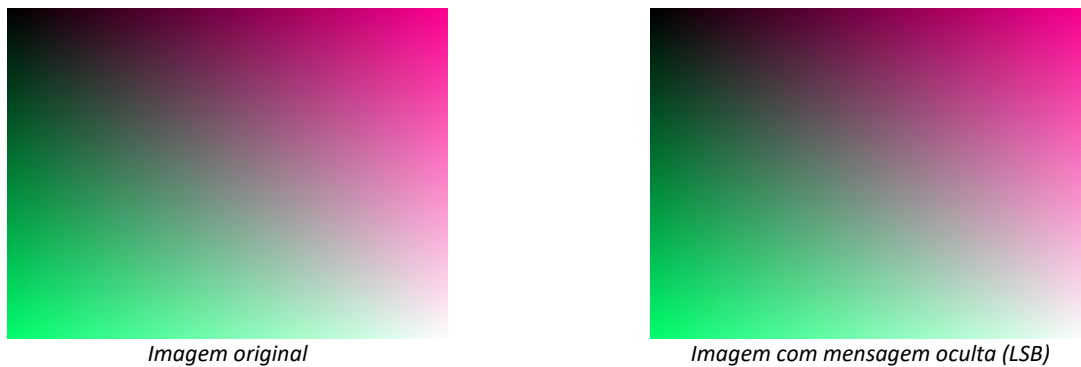


Figura 1 — Comparação visual entre a imagem original e a imagem com a mensagem oculta nos bits menos significativos. Nenhuma diferença é perceptível a olho nu.

5. Testes Automatizados

Para validar o funcionamento correto e a robustez dos três módulos, foram implementados 15 testes automatizados com `pytest`, cobrindo tanto os caminhos de sucesso quanto os casos de erro esperados:

- Hashing: cadastro bem-sucedido, rejeição de usuário duplicado, ausência da senha em texto puro no banco, hashes diferentes para a mesma senha (salt), login com senha correta/incorreta e usuário inexistente.
- Criptografia: cifrar/decifrar retorna o texto original, o texto cifrado não contém os dados em claro, decifragem com chave errada lança `InvalidTag`, e nonces diferentes a cada chamada.
- Esteganografia: esconder/extrair retorna a mensagem original, a imagem não é alterada visualmente (diferença máxima de 1 por canal), mensagem maior que a capacidade da imagem gera erro, e mensagens com acentuação/Unicode são preservadas.

Todos os 15 testes passaram com sucesso na execução final do projeto (`python -m pytest tests/ -v`).

6. Conclusão

A atividade permitiu aplicar, na prática, três pilares complementares da proteção de dados: o hashing com salt (para proteger segredos que nunca precisam ser recuperados, como senhas), a criptografia simétrica autenticada (para proteger dados que precisam ser recuperados integralmente por quem possui a chave) e a esteganografia (para ocultar a própria existência de uma informação). Cada técnica resolve um problema distinto e não deve ser confundida com as demais: um hash não pode ser "decifrado", uma cifra pressupõe que o atacante saiba que há uma mensagem, e a esteganografia, por si só, não substitui a criptografia caso o algoritmo de ocultação seja descoberto.

O uso de bibliotecas maduras e auditadas (`cryptography` e a biblioteca padrão `hashlib/secrets` do Python) foi proposital: implementações caseiras de primitivas criptográficas estão sujeitas a falhas sutis e não devem ser usadas em produção. Já a rotina de esteganografia, por não existir uma biblioteca padrão para LSB, foi implementada manualmente, o que exigiu manipulação direta de bits e bytes.

Repositório do código-fonte

O código completo, os testes automatizados e este relatório estão disponíveis no seguinte repositório GitHub:

<https://github.com/kauaenrico/ativ-3-seguranca-de-sistemas>